

得分



操作系统课程设计

题目： 32 位单核类 Unix 操作系统设计

姓 名	高康嘉
学 号	20231202051
院 系	网络空间安全与计算机学院
专 业	计算机科学与技术
指导教师	王煜

2026 年 1 月 2 日

目录

第 1 章 系统架构概述	1
1.1 内核模块划分	1
1.1.1 进程管理模块	1
1.1.2 内存管理模块	1
1.1.3 文件系统模块	2
1.2 启动流程分析	2
1.2.1 引导加载阶段	2
1.2.2 内核初始化阶段	2
第 2 章 进程与线程	3
2.1 进程控制块设计	3
2.1.1 进程状态转换	4
2.1.2 进程优先级管理	4
2.2 线程实现	4
2.2.1 内核线程	4
2.2.2 用户线程	4
第 3 章 代码高亮预览	5
3.1 综合示例	5

第 1 章 系统架构概述

本章将介绍操作系统的整体架构设计，包括内核模块划分、启动流程以及核心数据结构。操作系统是计算机系统中最重要软件之一，它负责管理硬件资源、提供用户接口以及运行应用程序。

1.1 内核模块划分

内核是操作系统的核心部分，负责直接与硬件交互。我们的操作系统采用模块化设计，将内核功能划分为以下几个主要模块。每个模块都有明确的职责和接口，便于维护和扩展。

1.1.1 进程管理模块

进程管理模块负责进程的创建、调度、同步和销毁。主要包括以下功能：

- 进程控制块（PCB）管理
- 进程调度算法实现
- 进程间通信机制
- 信号处理

进程是操作系统资源分配的基本单位，每个进程都有独立的地址空间和执行状态。

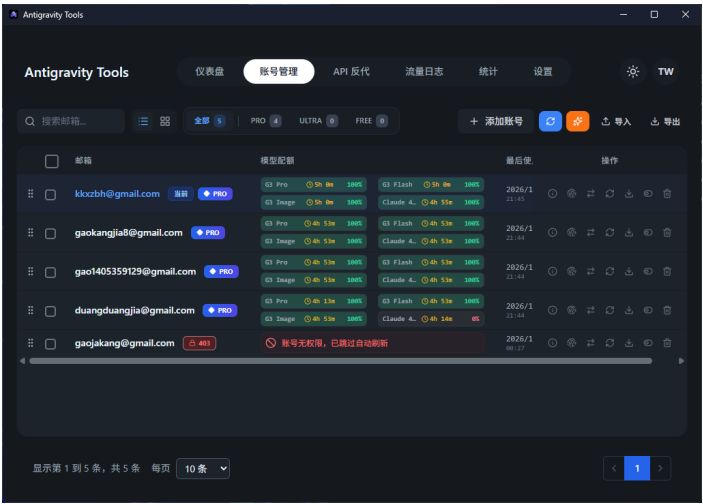


图 1.1: 第一章图片 A

1.1.2 内存管理模块

内存管理模块实现虚拟内存机制，包括页表管理、物理内存分配和页面置换算法。系统采用二级页表结构，支持 4KB 页面大小。

注意

内存分配时需要注意对齐要求，否则可能导致性能下降或硬件异常。

1.1.3 文件系统模块

文件系统模块提供对磁盘数据的抽象访问接口。我们实现了一个类 Unix 的简单文件系统，支持 inode 结构、目录层次和基本的文件操作。

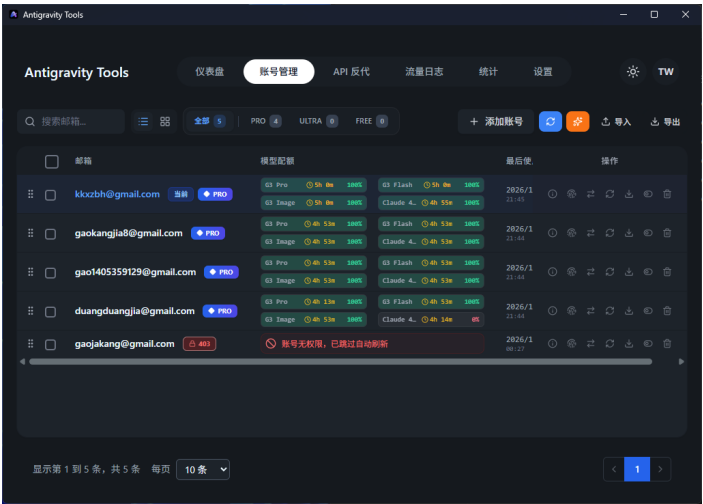


图 1.2: 第一章图片 B - 文件系统示意

1.2 启动流程分析

系统启动分为多个阶段，从 BIOS 自检到内核初始化完成。以下是启动流程的详细说明。

1.2.1 引导加载阶段

BIOS 完成自检后，将控制权交给引导扇区（MBR）。引导程序负责加载内核到内存并跳转执行。

1.2.2 内核初始化阶段

内核初始化包括：中断描述符表（IDT）设置、内存管理初始化、进程调度器初始化等。

第 2 章 进程与线程

本章详细介绍进程和线程的实现机制，包括数据结构设计和调度策略。

2.1 进程控制块设计

进程控制块（PCB）是描述进程状态的核心数据结构。表2.1列出了 PCB 的主要字段。

表 2.1 进程控制块主要字段

字段名	类型	说明
pid	uint32_t	进程标识符 ¹
status	enum	进程状态 ²
priority	uint8_t	进程优先级
pgdir	uint32_t*	页表指针
parent	PCB*	父进程指针

¹ 进程标识符在系统范围内唯一，由内核分配。

² 状态包括：READY、RUNNING、BLOCKED、ZOMBIE。

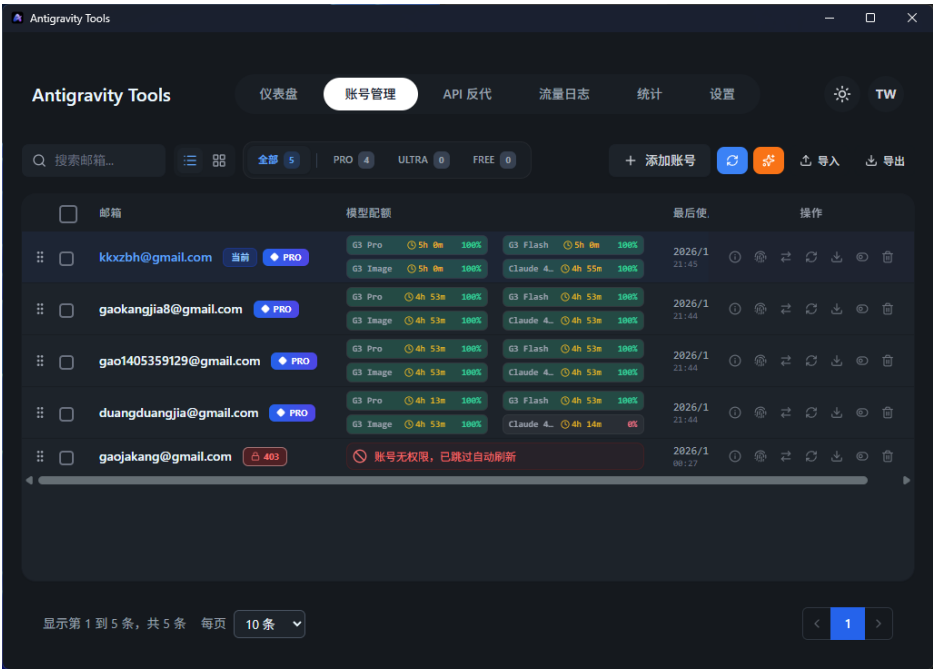


图 2.1: 测试图片

2.1.1 进程状态转换

进程在其生命周期中会经历多种状态：就绪、运行、阻塞和终止。状态转换由调度器和系统调用触发。

2.1.2 进程优先级管理

系统支持多级优先级调度，优先级范围为 0-31，数值越小优先级越高。

2.2 线程实现

线程是 CPU 调度的基本单位，多个线程可以共享同一进程的地址空间。

2.2.1 内核线程

内核线程运行在内核态，共享内核地址空间，用于执行系统任务。

2.2.2 用户线程

用户线程运行在用户态，通过系统调用请求内核服务。

第 3 章 代码高亮预览

本章展示 VS 主题的代码高亮效果，代码综合了常见的 C++ 语法元素。

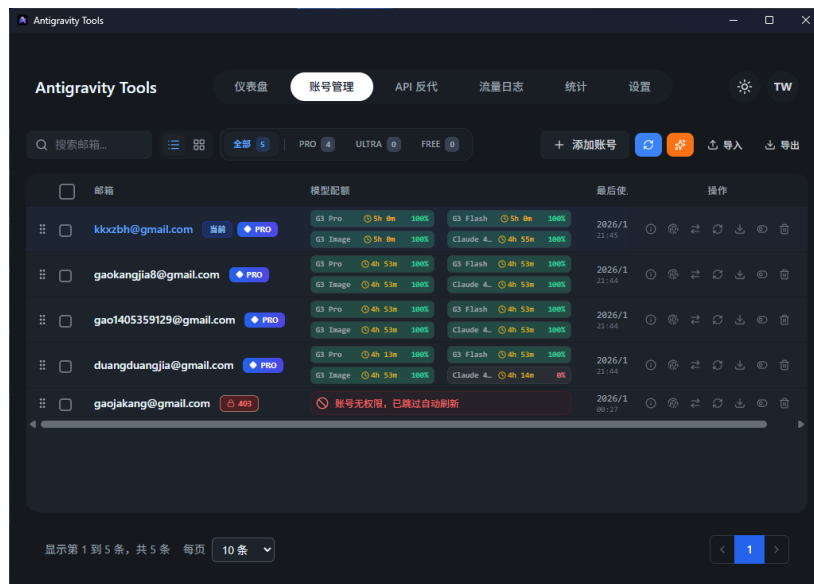


图 3.1: 第三章图片 - 代码结构

3.1 综合示例

```

1 | // 预处理指令 (Preprocessor)
2 | #include <iostream>
3 | #include <vector>
4 | #include <string>
5 | #define MAX_SIZE 1024
6 | #define DEBUG_MODE
7 |
8 | // 命名空间 (Namespace)
9 | namespace kernel {
10 |
11 | // 枚举类型 (Enum)
12 | enum class status : uint8_t {
13 |     ready    = 0,
14 |     running  = 1,

```

```

15 |     blocked = 2
16 | };
17 |
18 | // 类定义 (Class)
19 | class thread {
20 | public:
21 |     // 类型别名 (Type Alias)
22 |     using pid_t = uint32_t;
23 |
24 |     // 构造函数
25 |     explicit thread(const char* name, int priority = 31)
26 |         : name_(name), priority_(priority), pid_(next_pid_++) {}
27 |
28 |     // 成员函数 (Method)
29 |     auto run() -> void {
30 |         // 字符串 (String)
31 |         const char* msg = "Hello World";
32 |         std::string str = "Thread started";
33 |
34 |         // 数字 (Number)
35 |         int decimal = 42;
36 |         double pi = 3.14159;
37 |         uint32_t hex = 0xDEADBEEF;
38 |         size_t binary = 0b11010010;
39 |
40 |         // 运算符 (Operator)
41 |         int result = decimal + 10 * 2 - 5;
42 |         bool flag = (result > 0) && (priority_ <= 31);
43 |
44 |         // 控制流 (Control Flow)
45 |         if (flag) {
46 |             for (size_t i = 0; i < MAX_SIZE; ++i) {
47 |                 while (status_ != status::ready) {

```

```
48 |             // 等待
49 |         }
50 |     }
51 |     } else if (priority_ == 0) {
52 |         return;
53 |     }
54 |
55 |     // 标准库调用
56 |     std::cout << msg << std::endl;
57 |     std::vector<int> vec = {1, 2, 3, 4, 5};
58 | }
59 |
60 | // 模板函数 (Template)
61 | template<typename T>
62 | auto print(const char* fmt, T&& arg) -> uint32_t {
63 |     char buf[256]{};
64 |     auto len = std::format_to(buf, fmt, std::forward<T>(arg));
65 |     return static_cast<uint32_t>(len);
66 | }
67 |
68 | private:
69 |     const char* name_;
70 |     int priority_;
71 |     pid_t pid_;
72 |     status status_ = status::ready;
73 |     static inline pid_t next_pid_ = 1;
74 | };
75 |
76 | // 结构体 (Struct)
77 | struct bitmap {
78 |     uint8_t* bits;
79 |     size_t sz;
80 | }
```

```
81 |     auto test(size_t bi) -> bool {
82 |         return bits[bi / 8] >> (bi % 8) & 1;
83 |     }
84 |
85 |     auto set(size_t bi, bool value) -> void {
86 |         if (value) {
87 |             bits[bi / 8] |= 1 << (bi % 8);
88 |         } else {
89 |             bits[bi / 8] &= ~(1 << (bi % 8));
90 |         }
91 |     }
92 | };
93 |
94 | } // namespace kernel
95 |
96 | // 主函数
97 | int main() {
98 |     auto t = kernel::thread{"init", 0};
99 |     t.run();
100 |     return 0;
101 | }
```